

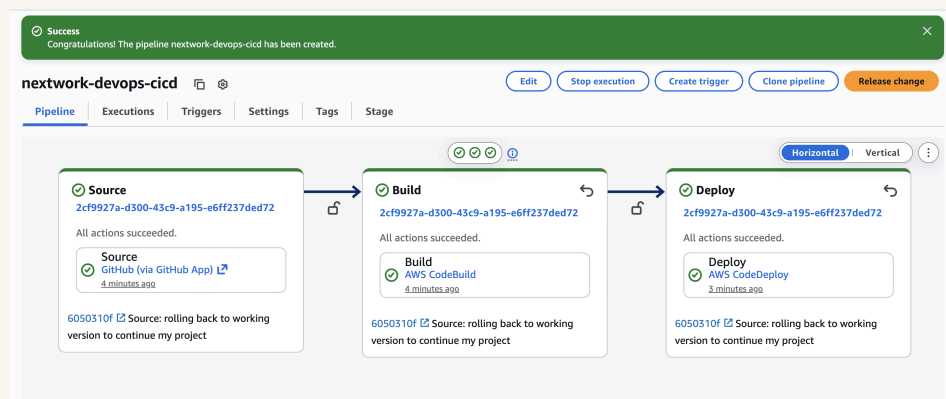


nextwork.org

Build a CI/CD Pipeline with AWS



sahini sourya





Introducing Today's Project!

In this project, I will demonstrate how to set up CI/CD pipeline with AWS CodePipeline. I'm doing this project to learn the automation of CI/CD that doesn't require manual updation for the deployment. Here, i can simply make the latest changes in code and CodePipeline will automate the deployment process all the way from github to CodeDeploy, finally this results to see the updated web application live in product which is not manualize the workflow in CodeBuild and CodeDeploy.

Key tools and concepts

Services I used were CodePipeline, CodeDeploy, CodeBuild, CodeArtifact, Github, EC2 instance, S3, IAM and CloudFormation. Key concepts I learnt include different stages in CI/CD pipeline, handling rollbacks and webhooks.

Project reflection

This project took me approximately 2 hours. The most challenging part was rolling back the deployment and resolve its corresponding its permission errors. It was most rewarding to see my live, deployed web app update without me having to build and deploy my project myself.



Starting a CI/CD Pipeline

AWS CodePipeline is a AWS Devops tool that helps me to create a workflow that automatically moves code from Github (my source code) all the way to CodeDeploy(my deployment tool). This helps me to make sure that deployments are consistent and reliable with less risk of human errors.

CodePipeline offers different execution modes based on how i treat multiple runs of same pipeline. I chose 'Superseded' which means i will focus on running the latest run that we start and cancel older ones. other options include 'Queued', which means runs are processed one after another including latest code will wait in queue while older runs are taking and 'Parallel' which means it allows multiple runs at the same time independent of each other.

A service role gets created automatically during setup so it can grant access to Codepipeline on other AWS services like S3 buckets, CodeBuild etc., and perform actions on my behalf.



Developer Tools

CodePipeline

► Source • CodeCommit

► Artifacts • CodeArtifact

► Build • CodeBuild

► Deploy • CodeDeploy

▼ Pipeline • CodePipeline

Getting started

Pipelines

Account metrics

Developer Tools

AWS CodePipeline

Visualize and automate the different stages of your software release process

AWS CodePipeline is a continuous integration and continuous delivery service for fast and reliable application and infrastructure updates. CodePipeline builds, tests, and deploys your code every time there is a code change, based on the release process models you define.

Create AWS CodePipeline pipeline

Get started with AWS CodePipeline by creating your first continuous delivery and continuous integration pipeline.

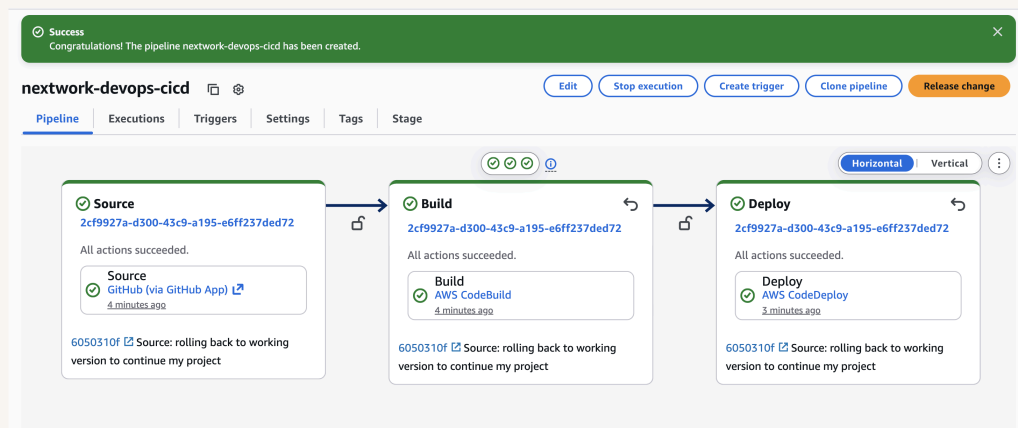
Create pipeline



CI/CD Stages

The three stages I've set up in my CI/CD pipeline are source (source code for my web project in Github), Build (building the web application in CodeBuild), Deploy (Deploying the changes to my web application using CodeDeploy).

CodePipeline organizes the three stages into a single diagram that showcases the flow of changes from source to deploy. In each stage, you can see more details on the pipeline execution that it belongs to, and the shortcuts to the connected service.





Source Stage

In the Source stage, the default branch tells CodePipeline to get code from our building version as there can be a lot of code versions in GitHub but some may use specific code version of some specific branches. In our case there is only branch (i.e., master).

The source stage is also where you enable webhook events, which let CodePipeline automatically start my pipeline whenever I push code into my source code repository in the specified branch (i.e., master). These events are like notifications or even-trigger that starts pipeline triggered by the change in source code. This makes to look my deployment CI/CD.

Add source stage [info](#)
Step 3 of 7

Source

Source provider
This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details.

GitHub (via GitHub App) ▼

Connection
Choose an existing connection that you have already configured, or create a new one and then return to this task.

arn:aws:codeconnections:ap- X or [Connect to GitHub](#)

Repository name
Choose a repository in your GitHub account.

Sourya-23/nextwork-web-project X

You can type or paste the group path to any project that the provided credentials can access. Use the format 'group/subgroup/project'.

Default branch
Default branch will be used only when pipeline execution starts from a different source or manually started.

master X

Output artifact format
Choose the output artifact format.

☒ **CodePipeline default**
AWS CodePipeline uses the default zip format for artifacts in the pipeline. Does not include Git metadata about the repository.

☐ **Full clone**
AWS CodePipeline passes metadata about the repository that allows subsequent actions to do a full Git clone. Only supported for AWS CodeBuild actions. [Learn more](#)



Build Stage

The Build stage sets up how i'll build my web application and make it ready for deployment. I configured CodeBuild to my build provider and to use the input artifact that was outputted by the source stage (CodeBuild). The input artifact for the build stage is the compressed code that resource stage retrieved from Github and compressed into a zip file.

Build - optional

Build provider
Choose the tool you want to use to run build commands and specify artifacts for your build action.

☐ Commands ☒ Other build providers

AWS CodeBuild

Project name
Choose a build project that you have already created in the AWS CodeBuild console. Or create a build project in the AWS CodeBuild console and then return to this task.

nextwork-devops-cicd or [Create project](#)

☐ Define buildspec override - optional
Buildspec file or definition that overrides the latest one defined in the build project, for this build only.

Environment variables - optional
Choose the key, value, and type for your CodeBuild environment variables. In the value field, you can reference variables generated by CodePipeline. [Learn more](#)

[Add environment variable](#)

Build type

☒ Single build
Triggers a single build.

☐ Batch build
Triggers multiple builds as a single execution.

Region
Asia Pacific (Mumbai)

Input artifacts
Choose an input artifact for this action. [Learn more](#)

SourceArtifact
Defined by: Source

☒ Enable automatic retry on stage failure



Deploy Stage

The Deploy stage is where i setup Codedeploy to be my deployment provider. It takes the build-artifact from CodeBuild, the application and deployment settings that i've defined in my deployment group.

Deploy - optional
Deploy provider
Choose how you want to deploy your application or content. Choose the provider, and then provide the configuration details for that provider.

AWS CodeDeploy

Region

Asia Pacific (Mumbai)

Input artifacts
Choose an input artifact for this action. [Learn more](#)

BuildArtifact ×
Defined by: Build
No more than 100 characters

Application name
Choose an application that you have already created in the AWS CodeDeploy console. Or create an application in the AWS CodeDeploy console and then return to this task.

Q nextwork-devops-cicd ×

Deployment group
Choose a deployment group that you have already created in the AWS CodeDeploy console. Or create a deployment group in the AWS CodeDeploy console and then return to this task.

Q nextwork-devops-cicd-deploymentgroup ×

☒ Configure automatic rollback on stage failure
☐ Enable automatic retry on stage failure



Success!

Since my CI/CD pipeline gets triggered by changes made in source code. I tested my pipeline by changing the code by adding a paragraph line "<p>If you see this line, that means your latest changes are automatically deployed into production by CodePipeline!</p>" to the index.jsp file.

The moment I pushed the code change the Codepipeline responded immediately by triggering to start new build and deployment again. The commit message under each stage reflects the latest code change that running pipeline is using, is the same commit message i've written for the new change indicating that updated code is the code taken to start deployment.

Once my pipeline executed successfully, I checked if the change is made into the live web application to confirm that pipeline is showing the expected the result; and yes the added new paragraph line is appeared in live web proving the changes went live into production straight away!



Hello World!

Welcome to the NextWork web project!

If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :o

If you see this line, that means your latest changes are automatically deployed into production by CodePipeline!

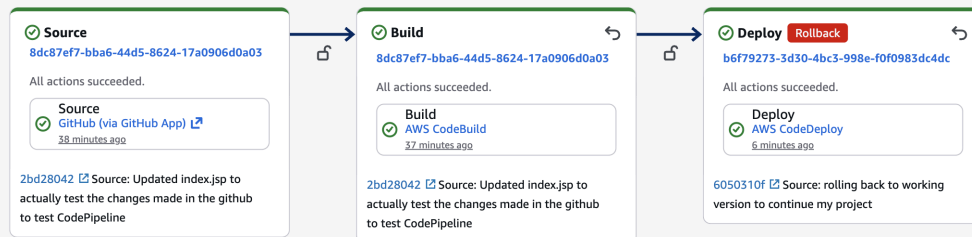


Testing the Pipeline

In a project extension, I initiated a rollback on the Deploy stage. Automatic rollback is important for to revert back to successful deployment if the changes made into source code cause problems or it doesn't look like it should be!

During the rollback, the source and build stage are unaffected because the changes before deployment. I could verify this by comparing the commit messages related to each stage - the Deploy stage commit message has reverted back to the first commit attached to my first pipeline run. The other two stages are still using my latest commit message.

After rollback, the live web app reverted back to its original state before i did any test for changes. This means the CodeDeploy roll back successfully and instantly too!





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

